

Neither Optional nor Free: DAG-based Byzantine Atomic Broadcast in the Context of State Machine Replication

Paul Bergmann ¹ and Maya Kuhlmann²

Byzantine atomic broadcast (BAB) ensures that all non-faulty replicas agree on the same total order of requests despite Byzantine failures. Classical leader-based BAB [CL99] centralizes both dissemination and ordering of requests in a single leader replica, making it a potential bottleneck. DAG-based BAB alleviates this by decoupling dissemination and ordering [Da22]. Replicas instead broadcast vertices in parallel, each referencing what they have previously heard from others, thus forming a DAG. The total order is then derived by having replicas agree on anchors in the DAG, e.g., through a common coin [Ke21].

DAG-based BAB has attracted considerable interest both in research [Ar25; Ba25; Sp22] and industry [Ap26; My26], particularly in distributed ledgers, which are state machine replication (SMR) [Sc90] systems: requests are executed deterministically and clients require a verifiable result. Nevertheless, prior publications mostly focus on efficient ordering, treating and evaluating the BAB protocol as standalone, decoupled from SMR. In this talk, we argue the opposite: for DAG-based BAB, the SMR layer is neither *optional* nor *free*.

SMR is not optional Bounding memory requires deleting vertices, yet replicas that missed a vertex will still request it, and these requests must be answered or agreement is lost. Prior work escapes this by offloading vertices to external storage, which lies outside the fault model and merely relocates the unboundedness. By examining BAB in the context of SMR, we observe that a lagging replica does not necessarily need the vertex, only its effect on the system state. SMR supplies this through certified checkpoints whose size depends on the application state rather than on the length of the DAG's history.

SMR is not free A client can only accept a result once it knows that a correct replica has executed its request. Establishing this canonically requires every replica to reply to every client: individual replies are not self-verifying, so the client must accumulate enough matching ones to conclude that a correct replica is among their senders. We show that this consumes exactly the throughput advantage that motivated DAG-based designs in the first place. To preserve that advantage, we integrate a threshold signature scheme into the DAG-based protocol so that a single reply suffices to certify execution.

Contribution Both the design and the evaluation of DAG-based BAB must therefore account for the SMR layer. We complete a DAG-based BAB protocol with our client interaction

¹ Friedrich-Alexander-Universität Erlangen-Nürnberg, Lehrstuhl für Systemsoftware, paul.bergmann@fau.de,  <https://orcid.org/0009-0006-1844-8446>

² Karlsruhe Institute of Technology, Forschungsgruppe Dezentrale Systeme und Netzdienste, unlwa@student.kit.edu

layer, evaluate the result end-to-end, and give a garbage collection scheme with provable memory bounds. We close on the methodological questions this raises about where a system ends and how we measure it.

References

- [Ap26] Aptos Foundation, <https://aptosnetwork.com>, 2026.
- [Ar25] Arun, B. et al.: Shoal++: High Throughput DAG BFT Can Be Fast and Robust! In: 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25). USENIX Association, 2025, <https://www.usenix.org/conference/nsdi25/presentation/arun>.
- [Ba25] Babel, K. et al.: Mysticeti: Reaching the Limits of Latency with Uncertified DAGs, 2025, arXiv: 2310.14821[cs], <http://arxiv.org/abs/2310.14821>.
- [CL99] Castro, M.; Liskov, B.: Practical Byzantine Fault Tolerance. In: 3rd Symposium on Operating Systems Design and Implementation (OSDI 99). USENIX Association, 1999, <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.
- [Da22] Danezis, G. et al.: Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In: Proceedings of the Seventeenth European Conference on Computer Systems. EuroSys '22, 2022, <https://doi.org/10.1145/3492321.3519594>.
- [Ke21] Keidar, I. et al.: All You Need is DAG. In: Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing. PODC'21, Association for Computing Machinery, pp. 165–175, 2021, <https://doi.org/10.1145/3465084.3467905>.
- [My26] Mysten Labs, <https://sui.io>, 2026.
- [Sc90] Schneider, F. B.: Implementing fault-tolerant services using the state machine approach: A tutorial. ACM Computing Surveys (CSUR) 22 (4), pp. 299–319, 1990.
- [Sp22] Spiegelman, A. et al.: Bullshark: DAG BFT Protocols Made Practical. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. CCS '22, 2022, <https://doi.org/10.1145/3548606.3559361>.